

TEACH YOURSELF LECTURE

Lecture 02

Computational Thinking & Algorithms

Chapter lecture — 24 topics with explanations, worked examples, Urdu notes, practice questions and a full answer key.

Computer Science XII · Bilingual support: English + اردو

کمپیوٹر سائنس — بارہویں جماعت: خود سیکھیں، خود پرکھیں

24 teach-yourself lectures in this pack

★ Golden topics, Urdu notes, review and answer key

XII

How to use this lecture

Learn this chapter **on your own**. Each topic is one A4 page. Read *Learn it*, try the worked example on paper, then answer *Check yourself* before looking at the answers. The chapter pack ends with a 75-mark exam. The green box repeats the idea in Urdu. ★ Golden topics are high-yield in board exams.

یہ لیکچر خود سیکھیں۔ مثال کاغذ پر حل کریں، پھر جواب دیکھنے سے پہلے خود کو پرکھیں۔

Lectures in this pack

Chapter 2: Computational Thinking & Algorithms. Study one lecture at a time, then use the chapter review and answer key at the end of this PDF.

L01. 2.1 Analyze algorithms for correctness

L03. 2.1.2 Stepwise Reasoning

L05. 2.2 Evaluating the Clarity of an Algorithm

L07. 2.2.2 Readability

L09. 2.3.1 Number of Steps (Big O Notation)

L11. 2.3.3 Number of Repetitions

L13. 2.4 Refinements to improve clarity and efficiency

L15. 2.5.1 Linear Data Structures: Array

L17. Linear Data Structures: Stack

L19. 2.5.2 Non-Linear Data Structures: Tree

L21. 2.5.3 Operations on Data Structures

L23. 2.7 Trace data retrieval in Array, List and Queue

L02. 2.1.1 Trace Tables

L04. 2.1.3 Comparing Trace Tables and Stepwise Reasoning

L06. 2.2.1 Modularity

L08. 2.3 Assess Algorithm Efficiency

L10. 2.3.2 Number of Conditions

L12. 2.3.4 The Efficiency Evaluation Framework

L14. 2.5 Concept of Data Structure

L16. Linear Data Structures: Linked List

L18. Linear Data Structures: Queue

L20. Non-Linear Data Structures: Graph

L22. 2.6 Identify data structures in problem scenarios

L24. Career Opportunities in Algorithms

Computational Thinking & Algorithms

Class XI taught you how to write algorithms. This chapter teaches you how to **judge** them: is the algorithm correct, clear and efficient? You will then learn the main **data structures** — array, linked list, stack, queue, tree and graph — so you can store data the smart way and trace how it is retrieved.

گیارہویں جماعت میں آپ نے الگورتھم لکھنا سیکھا۔ اس باب میں آپ سیکھیں گے کہ الگورتھم درست، واضح اور تیز ہے یا نہیں، اور ایرے، لنکڈ لسٹ، سٹیک، کیو، ٹری اور گراف جیسے ڈیٹا سٹرکچرز میں ڈیٹا کیسے رکھا اور نکالا جاتا ہے۔

What you will learn

- Define **correctness** and prove it with a complete **trace table**.
- Evaluate **clarity** using modularity (SRP, coupling, abstraction) and readability (names, flow, consistency).
- Explain how the number of **conditions** and **repetitions** changes efficiency, and apply the three-question efficiency framework.
- Define a **data structure** and classify it as linear or non-linear.
- Draw a tree and a graph, and name the five operations on data structures.
- Apply **stepwise reasoning** and compare it with a trace table on at least four points.
- Define **efficiency** in time and space, and state Big O for $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$ and $O(n^2)$ with examples.
- Refine an algorithm for clearer names, modules, comments and fewer wasted steps.
- Draw and trace an array, linked list, stack (PUSH/POP) and queue (ENQUEUE/DEQUEUE).
- Choose a data structure for a real scenario and trace retrieval in an array, a linked list and a queue.

2.1 Analyze algorithms for correctness

LEARN IT

An algorithm is **correct** if its **output matches the expected results** for a given input. Analysing correctness means showing that the algorithm produces the intended output for **all valid inputs**, not just one lucky example.

Two methods are used in this course:

Trace tables



paper execution, line by line



Stepwise reasoning



logic blocks, overall state

You already met the idea of a dry run in Class XI. Here you learn to do it formally, and to choose the method that fits the size of the algorithm.

WORKED EXAMPLE

Worked example: what "correct" means. An algorithm is supposed to count how many of the heights 1.91, 2.01, 1.87, 1.57, 1.48 lie between 1.57 m and 1.91 m inclusive.

Expected output: **3** (1.91, 1.87 and 1.57 qualify; 2.01 is too tall; 1.48 is too short).

If a program prints 4, it **runs** but it is **not correct**. Correctness is about the right answer, not about "no crash".

REMEMBER IT

Remember: **runs ≠ correct**. A logical error still produces output — just the wrong output. That is why we analyse correctness on paper before we trust the code.

COMMON MISTAKE

Mistake: testing only one "normal" input and calling the algorithm correct.

Fix: also try edge cases (empty list, $N = 0$, the exact boundary values 1.57 and 1.91). The definition says **all valid inputs**.

اردو میں سمجھیں

الگورتھم تب درست (Correct) سمجھا جاتا ہے جب دیے گئے ان پٹ پر اس کا آؤٹ پٹ وہی ہو جو ہونا چاہیے۔ صرف چل جانا کافی نہیں؛ منطقی غلطی سے غلط جواب بھی نکل سکتا ہے۔ درستگی جانچنے کے دو طریقے ہیں: ٹریس ٹیبل (Trace Table) اور مرحلہ وار استدلال (Stepwise Reasoning)۔

CHECK YOURSELF

1. When is an algorithm considered correct?
2. Name the two methods used to analyse correctness in this chapter.
3. Why is "the program ran without an error" not enough?

Answers: 1. When its output matches the expected results for a given input (and does so for all valid inputs). 2. Trace tables and stepwise reasoning. 3. Because a logical error can still produce output — the wrong output.

2.1.1 Trace Tables ★ Golden

LEARN IT

A **trace table** is a technique used to monitor how the values of variables change during the execution of an algorithm. It is a **paper execution** of the code — a manual walkthrough, also called a **dry run**. You act as the computer and fill in one row after another.

Purpose of trace tables

- **Dry running:** execute the algorithm line by line by hand.
- **Debugging:** find logical errors where the algorithm runs but the output is wrong.
- **Validation:** check that the algorithm does what it is intended to do.

Components of a trace table

- **Line / step number** — which line is being executed.
- **Variables** — one column for every variable.
- **Input / output** — user inputs and printed results.
- **Condition check** — True/False for IF, WHILE, FOR, and similar tests.

WORKED EXAMPLE

Worked example: soldiers who meet the height rule. To qualify for training, a soldier's height must be between **1.57 and 1.91 metres, inclusive**.

Inputs: all soldiers' heights. Output: the count of eligible soldiers.

Step 1: Start
Step 2: SET Total = 0
Step 3: Set AllHeights = [1.91, 2.01, 1.87, 1.57, 1.48]
Step 4: FOR EACH height FROM AllHeights DO
Step 5: IF height >= 1.57 AND height <= 1.91 THEN
Step 6: SET Total = Total + 1
Step 7: END IF
Step 8: END FOR EACH
Step 9: Print Total
Step 10: End

Inclusive means the two boundary values **count**: 1.57 and 1.91 are eligible.

Step	Total	Height	AllHeights (input)	Height between 1.57 and 1.91?	Output
2	0				
3	0		1.91, 2.01, 1.87, 1.57, 1.48		
4	0	1.91			
5	0	1.91		True	
6	1	1.91			
4	1	2.01			
5	1	2.01		False (skip Step 6)	
4	1	1.87			
5	1	1.87		True	
6	2	1.87			
4	2	1.57			
5	2	1.57		True (boundary)	
6	3	1.57			
4	3	1.48			
5	3	1.48		False (skip Step 6)	
9	3				3

Eligible: 1.91, 1.87, 1.57. Not eligible: 2.01 (above 1.91) and 1.48 (below 1.57). Printed Total = 3.

```
AllHeights = [1.91, 2.01, 1.87, 1.57, 1.48]
Total = 0
for height in AllHeights:
    if height >= 1.57 and height <= 1.91:
        Total = Total + 1
print(Total)
```

OUTPUT
3

REMEMBER IT

How to fill a trace table without getting lost. Write a new row every time a variable **changes** or a condition is **tested**. Copy unchanged values into the new row so you never guess. Circle the final output.

Mnemonic: **V-I-C-O** — Variables, Input, Condition, Output. If a column is missing, the table is incomplete.

COMMON MISTAKE

Mistake 1: treating 1.57 or 1.91 as out of range. The words "inclusive" and the operators $\geq$ and $\leq$ mean the boundaries **are allowed**.

Mistake 2: updating Total on a False condition. If the IF is False, skip the SET line and leave Total unchanged.

Mistake 3: writing only the last Total. Examiners award marks for the **changing values in the table**, not only for the printed 3.

اردو میں سمجھیں

ٹریس ٹیبل الگورتھم کا کاغذی اجرا (Dry Run) ہے۔ ہر قطار میں لکیر نمبر، متغیرات کی نئی قیمتیں، ان پٹ/آؤٹ پٹ اور شرط کا نتیجہ (True/False) لکھا جاتا ہے۔ فوجیوں کی مثال میں قد 1.57 سے 1.91 میٹر تک ہونا چاہیے، حدود سمیت؛ پانچ قدوں میں سے تین اہل ہیں اس لیے آؤٹ پٹ 3 ہے۔

EXAM TIP

Very common long question: "Dry-run the given algorithm using a trace table and state the output." Draw columns for **every** variable plus a condition column. Show the loop once per item. Boundary values ($\geq$, $\leq$, empty list) are favourite traps. Short questions ask: purpose of a trace table (dry run / debug / validate) and its components (line, variables, I/O, condition).

CHECK YOURSELF

1. List the four usual columns / components of a trace table.
2. In the soldier example, why is height 1.57 counted but 1.48 not counted?
3. What is printed, and how many times is Step 6 executed?

Answers: 1. Line/step number, variables, input/output, condition check. 2. 1.57 is the lower boundary (inclusive, so True); 1.48 is below 1.57 (False). 3. 3 is printed; Step 6 runs three times (for 1.91, 1.87 and 1.57).

2.1.2 Stepwise Reasoning

LEARN IT

Stepwise reasoning (also called **logic verification**) breaks an algorithm into logical sections and checks the **state** of the algorithm at each stage. A **state** is a snapshot of what is true about the variables at that moment (for example, “Sum currently holds the total of the numbers seen so far”).

You do **not** fill a row for every line. You think through the logic:

1. Analyse each step of the algorithm.
2. Check: does it **start in a valid state**?
3. Does it **handle all possible cases**?
4. Are the **conditions correct**?
5. If the loop is correct at the **start** of an iteration, does it remain correct at the **end**?

WORKED EXAMPLE

Worked example: sum of the numbers from 1 to N.

Step 1: Start
Step 2: Input N
Step 3: SET Sum = 0
Step 4: For i = 1 to N
Step 5: Sum = Sum + i
Step 6: END For
Step 7: Print Sum
Step 8: End

Stepwise reasoning breakdown

Check	What you verify
Initial state	Sum = 0 before the loop. Valid: nothing has been added yet.
Loop boundary	The loop runs exactly from i = 1 to i = N. No missing number, no extra number.
Iteration	Each i is added once: after k iterations, Sum = 1+2+...+k.
Final state	When the loop ends, Sum holds 1+2+...+N, which is the required total.
Edge case N = 0	The loop body never runs, Sum stays 0. Correct (the sum of no numbers is 0).
Check case N = 5	Sum becomes 15, which matches $5 \times 6 / 2 = 15$.

```
N = 5
Sum = 0
for i in range(1, N + 1):
    Sum = Sum + i
print(Sum)
```

OUTPUT
15

```
N = 0
Sum = 0
for i in range(1, N + 1):
    Sum = Sum + i
print(Sum)
```

OUTPUT
0

REMEMBER IT

Think of stepwise reasoning as a **teacher checking a proof**, not as a cashier counting coins one by one. You ask “is this block always true?” instead of “what is Sum when i = 17?”.

COMMON MISTAKE

Mistake: rewriting the whole trace table and calling it stepwise reasoning.

Fix: write short comments about **state** (start, loop, finish, edges). If you listed every i from 1 to 1000, you used the wrong method.

اردو میں سمجھیں

مرحلہ وار استدلال میں الگورتھم کو منطقی حصوں میں توڑ کر ہر مرحلے پر اس کی حالت (State) دیکھی جاتی ہے: کیا شروع درست ہے، کیا تمام صورتیں سنبھلی ہیں، کیا شرط صحیح ہے، اور کیا لوپ کے ہر چکر کے بعد بھی نتیجہ درست رہتا ہے۔ ایک سے N تک جمع کی مثال میں Sum شروع میں 0 ہے، لوپ 1 سے N تک چلتا ہے، اور N=0 پر جواب 0 ہی رہتا ہے۔

CHECK YOURSELF

1. What other name is given to stepwise reasoning?
2. For the sum algorithm, why is Sum = 0 a valid start?
3. What happens when N = 0, and is that correct?

Answers: 1. Logic verification. 2. Because no numbers have been added yet, so the running total should be zero. 3. The loop does not run and Sum remains 0, which is the correct sum of no numbers.

2.1.3 Comparing Trace Tables and Stepwise Reasoning

★ Golden

LEARN IT

Both methods verify correctness, but they fit different situations depending on the **complexity and length** of the algorithm.

Feature	Trace table	Stepwise reasoning
Best for	Small algorithm blocks, complex loops, and beginners learning to track variables.	Large systems, architectural logic, and abstract mathematical proofs.
Effort required	High. Very tedious and impractical if a loop runs 1,000 times.	Moderate. Needs strong logical deduction and abstract thinking.
Primary goal	Catching syntax misinterpretations or specific arithmetic errors line by line.	Catching conceptual or structural flaws in the overall design.
Level	Micro-level (line by line). Checks exact values.	Macro-level (logic blocks). Checks overall state.

Trace table

Micro-level
Line by line
Exact values

Stepwise reasoning

Macro-level
Logic blocks
Overall state

Trace tables zoom in on values; stepwise reasoning zooms out on design.

WORKED EXAMPLE

Worked example: which method would you pick?

1. **A 12-line loop that adds five heights.** Use a **trace table**. The loop is short, you are a beginner, and you need the exact Total.
2. **A billing system with 40 functions.** Use **stepwise reasoning**. You cannot fill 10,000 rows. You check each module's start state, end state and edge cases.
3. **A loop from $i = 1$ to 1,000 that should produce Sum = 500500.** Do **not** write 1,000 rows. Argue: "Sum starts at 0; each i is added once; final Sum = $n(n+1)/2 = 1000 \times 1001/2 = 500500$." That is stepwise reasoning. Then dry-run a tiny N (say 5) with a trace table if you want extra confidence.

REMEMBER IT

Micro vs macro. Trace table = microscope (every cell). Stepwise reasoning = map (whole city). Beginners start with the microscope; designers need the map.

COMMON MISTAKE

Mistake: saying one method is "better" in every question.

Fix: they have different jobs. For a short dry-run in the exam, draw the table. For "explain why the algorithm is correct for any N ", write stepwise reasoning.

اردو میں سمجھیں

ٹریس ٹیبل خرد بینی (Micro) سطح پر کام کرتا ہے: ہر لائن اور ہر قیمت۔ مرحلہ وار استدلال بڑی (Macro) سطح پر ڈیزائن اور حالت چیک کرتا ہے۔ چھوٹے لوپ اور میتدیوں کے لیے ٹیبل بہتر ہے؛ ہزار بار چلنے والے لوپ یا بڑے سسٹم کے لیے استدلال بہتر ہے۔

EXAM TIP

Differentiate questions are almost guaranteed: "Compare a trace table with stepwise reasoning." Write a **four-row table** (best for, effort, primary goal, micro/macro). Add one sentence of advice: table for small loops and exact values; reasoning for large systems and proofs. Do not write "trace table is always better".

CHECK YOURSELF

1. Which method is micro-level and checks exact values?
2. Why is a trace table a poor choice if a loop runs 1,000 times?
3. Which method is better for catching a structural flaw in a large design?

Answers: 1. Trace table. 2. It is very tedious and impractical (too many rows). 3. Stepwise reasoning (macro-level, conceptual / structural flaws).

2.2 Evaluating the Clarity of an Algorithm

LEARN IT

Clarity means an algorithm is easy to **read, understand and modify**. In many cases, evaluating clarity is even more important than making the algorithm slightly faster. A clear algorithm is easier to **maintain, debug and extend** over time.

The two key aspects of clarity are:

Clarity → Modularity (parts) → Readability (text and flow)

WORKED EXAMPLE

Worked example: speed versus clarity. Algorithm A finds the class average in 8 confusing lines with variables a , b , c . Algorithm B uses two named functions and takes the same number of additions. Both are $O(n)$. Algorithm B is the better school solution because the next person (or you, in three months) can change it without breaking it.

REMEMBER IT

Ask two questions: **"Can I split this into parts?"** (modularity) and **"Can another student read it aloud and understand it?"** (readability).

COMMON MISTAKE

Mistake: treating a clever one-line trick as "better" than a clear five-line solution.

Fix: the source says clarity is often more important than speed. Board answers should praise meaningful names and modules, not compacted code.

اردو میں سمجھیں

وضاحت (Clarity) کا مطلب ہے الگورتھم پڑھنا، سمجھنا اور بدلنا آسان ہو۔ بہت سی صورتوں میں وضاحت رفتار سے بھی اہم ہے۔ اس کے دو حصے ہیں: ماڈیولریٹی (Modularity) یعنی کام کو حصوں میں بانٹنا، اور پڑھائی پن (Readability) یعنی متن اور بہاؤ صاف ہونا۔

CHECK YOURSELF

1. What does clarity mean?
2. Name the two key aspects of clarity.
3. Why can clarity matter more than a small speed gain?

Answers: 1. The algorithm is easy to read, understand and modify. 2. Modularity and readability. 3. Because a clear algorithm is easier to maintain, debug and extend over time.

2.2.1 Modularity

LEARN IT

Modularity means breaking an algorithm into smaller, independent parts such as **functions, classes or modules**. This makes the algorithm more organised and easier to work with.

How to evaluate modularity

1. **Single Responsibility Principle (SRP)**. Each module should focus on **one** specific task. If a function calculates a sum, prints a report and saves a file, it violates SRP.
2. **Coupling**. Modules should be reusable in different contexts. Good modularity means **loose coupling**: a change in one part does not smash the others. Tight coupling is when two functions share so many hidden details that you cannot reuse one without the other.
3. **Abstraction**. A well-designed algorithm hides complex implementation details. You can call a sorting function without knowing whether it uses Bubble Sort or Selection Sort inside.

WORKED EXAMPLE

Worked example: average of an array.

Poor modularity (monoblock)

All logic sits in one block, so you cannot reuse the sum.

```
Step 1: Start
Step 2: Set A = [1, 2, 3, 4, 5]
Step 3: Set n = 5
Step 4: Set total = 0
Step 5: Set avg = 0
Step 6: For I = 1 to n
Step 7: total = total + A[I]
Step 8: End For
Step 9: avg = total / n
Step 10: Print avg
Step 11: End
```

Good modularity (functions)

Two clear functions: CalculateSum() and FindAverage().

```
Step 1: Start
Step 2: Set A = [1, 2, 3, 4, 5]
Step 3: Set n = 5
Step 4: Print FindAverage(A, n)
Step 5: End

Function FindAverage(A, n)
    Sum = CalculateSum(A, n)
    Avg = Sum / n
    Return Avg
End Function

Function CalculateSum(A, n)
    Set Total = 0
    For I = 1 to n
        Total = Total + A[I]
    End For
    Return Total
End Function
```

SRP check: CalculateSum only adds; FindAverage only divides. Loose coupling: you can reuse CalculateSum to find a total without computing an average. Abstraction: the main algorithm just calls FindAverage and does not see the loop.

The source pseudocode numbers array positions from 1. Python numbers them from 0, so the loop is `for i in range(n)`.

```
def calculate_sum(A, n):
    total = 0
    for i in range(n):
        total = total + A[i]
    return total

def find_average(A, n):
    return calculate_sum(A, n) / n

A = [1, 2, 3, 4, 5]
n = 5
print(find_average(A, n))
```

OUTPUT
3.0

REMEMBER IT

SRP in one line: **one function, one job**. If you cannot name the function with a short verb-phrase (CalculateSum, FindAverage), it is doing too much.

COMMON MISTAKE

Mistake: copying the 1-based loop `for I = 1 to n: total = total + A[I]` into Python. Then `A[1]` skips the first item and `A[5]` crashes.

Fix: in Python use `for i in range(n)` so the indices are 0, 1, 2, 3, 4.

اردو میں سمجھیں

ماڈیولریتی کا مطلب الگورتھم کو آزاد حصوں (فنکشن، کلاس، ماڈیول) میں توڑنا ہے۔ جانچ کے تین اصول: واحد ذمہ داری (SRP) یعنی ایک فنکشن ایک کام؛ ڈھیلی جڑت (Loose Coupling) تاکہ ایک حصہ بدلنے سے دوسرے نہ ٹوٹیں؛ اور تجرید (Abstraction) یعنی اندرونی تفصیل چھپا کر صرف کام منگوانا، جیسے سورت فنکشن بلانے بغیر یہ جانے کہ بل ہے یا سلیکشن۔

CHECK YOURSELF

1. What does SRP say each module should do?
2. In the good average algorithm, which function hides the loop?
3. Why is the monoblock harder to reuse?

Answers: 1. One specific task. 2. CalculateSum (FindAverage only divides; the main program only prints). 3. All logic is in one block, so you cannot call "just the sum" from another algorithm.

2.2.2 Readability

LEARN IT

Readability is how easily another person can understand the purpose and logic of an algorithm **without running it**. It is about how clear the text and the flow are.

How to evaluate readability

1. **Naming conventions** — consistent rules for names. Good names make an algorithm easier to search and understand.
Bad: $v = d / t$
Good: `velocity = distance / time`
2. **Self-documentation** — the algorithm explains itself, so you need fewer comments. Meaningful names and clean formatting make the purpose obvious.
3. **Control-flow simplicity** — keep the logic straightforward. Avoid deep nesting (IF inside IF inside IF), limit extra conditions, and keep each function on one task.
4. **Consistency** — one style throughout: indentation, spacing, and casing (camelCase or snake_case). Then the logic stands out instead of the mess.

Names → Self-doc → Simple flow → Consistent

WORKED EXAMPLE

Worked example: the same formula, two styles.

```
# Hard to read
d = 100
t = 4
s = 20
v = d / t
if v > s:
    if t > 0:
        print(v)
```

OUTPUT

25.0

```
# Readable
distance = 100
time = 4
velocity = distance / time
speed_limit = 20
if time > 0 and velocity > speed_limit:
    print(velocity)
```

OUTPUT

25.0

The second version uses names, avoids an IF nested inside an IF, and keeps one consistent snake_case style. You can understand it without running it. Both print 25.0, but only the second is clear.

REMEMBER IT

Remember readability as **N-S-S-C**: **N**ames, **S**elf-documentation, **S**imple flow, **C**onsistency.

COMMON MISTAKE

Mistake: dumping a comment on every line (`total = total + 1 # add one to total`).

Fix: if the name already says it, skip the comment. Comments should explain **why**, not repeat **what**.

اردو میں سمجھیں

پڑھائی پن یہ ہے کہ دوسرا شخص الگورتھم چلائے بغیر اس کا مقصد سمجھ لے۔ اچھے نام (`velocity = distance / time`)، خود بیان (کم تبصرے)، سادہ کنٹرول فلو (گہری IF کے بغیر) اور ایک جیسا انداز (casing اور indentation) اسے آسان بناتے ہیں۔

CHECK YOURSELF

1. Rewrite $v = d / t$ with readable names.
2. What is deep nesting, and why is it a readability problem?
3. Name two style habits that support consistency.

Answers: 1. `velocity = distance / time`. 2. IF inside IF inside IF (or similar); it makes the flow hard to follow. 3. Any two of: consistent indentation, spacing, and casing (camelCase or snake_case).

2.3 Assess Algorithm Efficiency

LEARN IT

Algorithm efficiency is measured in terms of **time** and **space** — how fast it runs and how much memory it uses.

The key considerations for assessing efficiency are:

1. **Number of steps** (described with **Big O notation**).
2. **Number of conditions** (IF decisions).
3. **Number of repetitions** (loops).

Time (speed) + Space (memory) → Efficiency

WORKED EXAMPLE

Worked example: two ways to find a name in 1,000,000 sorted records.

Linear search checks about 1,000,000 names in the worst case (time grows with n). Binary search checks about 20 names (it halves the list each time). Both can be correct and clear. They are not equally efficient.

REMEMBER IT

Efficiency is always “as the input grows”. An algorithm that feels instant on 10 items can freeze on 10 million items if it is $O(n^2)$.

COMMON MISTAKE

Mistake: judging efficiency by how short the code looks.

Fix: a five-line nested loop can do n^2 work. Count steps, conditions and repetitions — not lines of text.

اردو میں سمجھیں

کارکردگی (Efficiency) وقت (رفتار) اور جگہ (میموری) سے ناپی جاتی ہے۔ تین چیزیں دیکھی جاتی ہیں: قدموں کی تعداد (بگ او)، شرطوں کی تعداد، اور دہرائی (لوپ) کی تعداد۔ چھوٹے ان پٹ پر تیز لگنے والا الگورتھم بڑے ان پٹ پر سست پڑ سکتا ہے۔

CHECK YOURSELF

1. Efficiency is measured in terms of which two resources?
2. List the three key considerations named in the source.

Answers: 1. Time (speed) and space (memory). 2. Number of steps (Big O), number of conditions, number of repetitions.

2.3.1 Number of Steps (Big O Notation) ★ Golden

LEARN IT

When we discuss efficiency we ask how performance changes as the **input size n** increases. **Big O notation** is a mathematical method used to describe how the running time (or memory) **grows** as n grows. We ignore small constants and keep the most important term: if an algorithm does $3n + 5$ steps we write $O(n)$. In computer science, **log n** almost always means log base 2 (how many times you can halve n until you reach 1).

Five common classes

Class	Name	What happens when n grows	Typical example	Grade
$O(1)$	Constant	Time stays the same for 10 items or 10 million.	Access <code>arr[8]</code> (or any known index).	Excellent
$O(\log n)$	Logarithmic	Time grows very slowly; the problem is halved each step.	Binary search on a sorted list.	Great
$O(n)$	Linear	If n doubles, time roughly doubles.	Linear search of an unsorted list.	Fair / Good
$O(n \log n)$	Linearithmic	A little worse than linear; much better than n^2 . Common in good sorting.	Merge Sort (split in half, then merge).	Good
$O(n^2)$	Quadratic	Time grows with the square of n . Nested loops. Painful for large n .	Bubble Sort; compare every item with every other item.	Poor

The source stresses $O(1)$, $O(\log n)$, $O(n)$ and $O(n^2)$. $O(n \log n)$ is the class of efficient general-purpose sorts, so you should recognise it when a question says "better than Bubble Sort but more than linear".

Growth table (log is $\log_2$)

Each cell is the approximate number of operations for that n . Read down a column to see how fast that class explodes.

n	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$
2	1	1	2	2	4
4	1	2	4	8	16
8	1	3	8	24	64
16	1	4	16	64	256
32	1	5	32	160	1,024
1,024	1	10	1,024	10,240	1,048,576
1,000,000	1	~20	1,000,000	~20,000,000	1,000,000,000,000

From $n = 8$ to $n = 1,000,000$, $O(1)$ does not move. $O(\log n)$ only goes from 3 to about 20. $O(n)$ becomes a million, and $O(n^2)$ becomes a million million — which is why nested loops "fail for large datasets".



Same $n = 16$. Quadratic work is already four times $n \log n$ and sixteen times linear.

WORKED EXAMPLE

Worked example A — $O(1)$: first element of an array. Whether the array holds 10 or 10 million values, `arr[0]` is one jump to a known address.

```
arr = [23, 76, 12, 90, 34, 68]
print(arr[0])
```

OUTPUT
23

Worked example B — $O(n)$: linear search. Each extra item adds one extra check.

```
def linear_search(items, key):
    for i, value in enumerate(items):
        if value == key:
            return i
    return -1

print(linear_search([10, 50, 20, 70, 90, 80, 20])
print(linear_search([10, 50, 20, 70, 90, 80, 99])
```

OUTPUT
2
-1

Worked example C — $O(\log n)$: binary search on a sorted list. Each step throws away half of the remaining items. For 8 sorted numbers the worst case is about 4 comparisons (floor($\log_2 8$) + 1).

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1
    steps = 0
    while low <= high:
        mid = (low + high) // 2
        steps += 1
        if arr[mid] == key:
            return mid, steps
        if arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1, steps

data = [3, 8, 12, 17, 23, 31, 46, 52]
print(binary_search(data, 40))
print(binary_search(data, 3))
```

OUTPUT
(6, 3)
(0, 3)

Key 40 is found at index 6 after 3 comparisons. Key 3 is found at index 0 after 3 comparisons. Never more than a handful of steps, even if the list later grows to a million (about 20 steps).

Worked example D — $O(n \log n)$: Merge Sort idea. To sort 8 items, Merge Sort makes $\log_2 8 = 3$ split-levels, and each level walks all 8 items once, so work = $8 \times 3 = 24$, which matches the growth table. Bubble Sort on 8 items is on the order of $8^2 = 64$ comparisons. That gap becomes enormous at $n = 1,000,000$.

Worked example E — $O(n^2)$: nested loops.

```
n = 4
count = 0
for i in range(n):
    for j in range(n):
        count += 1
print(count)
```

OUTPUT
16

Inner body runs $n \times n = 16$ times. That is the pattern of Bubble Sort and of "compare every item with every other item".

REMEMBER IT

Order from fastest to slowest (for large n): $O(1) \rightarrow O(\log n) \rightarrow O(n) \rightarrow O(n \log n) \rightarrow O(n^2)$

Mnemonic: "Constant Logs Never Need Squares" — Constant, Logarithmic, N, N-log-N, Squares.

COMMON MISTAKE

Mistake 1: writing $O(n)$ for binary search because "there is a loop". The loop runs about log n times, not n times.

Mistake 2: writing $O(2n)$ or $O(n+5)$. Big O drops constants: both are $O(n)$.

Mistake 3: using binary search on an unsorted list. Binary search is $O(\log n)$ only on sorted data.

اردو میں سمجھیں

یگ او (Big O) بتاتا ہے کہ ان پٹ n بڑھنے پر وقت یا میموری کیسے بڑھے گی۔ $O(1)$ مستقل (ایرے کا `arr[0]`)، $O(\log n)$ پر قدم پر آندا (بائنری سرچ)، $O(n)$ تکرری (لینیئر سرچ)، $O(n \log n)$ اچھے سارٹ جیسے مرچ سارٹ، اور $O(n^2)$ مرچ (جمل سارٹ، اندرونی لوپ)۔ بڑے n پر $O(n^2)$ ناقابل استعمال ہو جاتا ہے۔

EXAM TIP

Mnemonic the five rows of the class table (name, growth, one example, grade). Typical MCQ: "Accessing `arr[0]` is ...?" $\rightarrow O(1)$. "Binary search is ...?" $\rightarrow O(\log n)$. "Nested loops / Bubble Sort ...?" $\rightarrow O(n^2)$. Long questions ask you to fill or interpret a growth table, or to say what happens if n becomes 10 times larger (linear $\rightarrow 10 \times$ time; quadratic $\rightarrow 100 \times$ time). Always mention that log n is "halving" and n^2 is "nested loops".

CHECK YOURSELF

- Give one example each of $O(1)$, $O(\log n)$, $O(n)$ and $O(n^2)$.
- From the growth table, how many operations does $O(n^2)$ need when $n = 32$?
- Why is Merge Sort described as $O(n \log n)$?

Answers: 1. $O(1)$ `arr[0]`; $O(\log n)$ binary search; $O(n)$ linear search; $O(n^2)$ Bubble Sort / nested loops. 2. 1,024. 3. It makes about log n split-levels and does about n work on each level, so $n \times \log n$.

2.3.2 Number of Conditions

LEARN IT

Conditions (IF statements) are decision points. Every condition forces the CPU to evaluate True or False, so excessive branching reduces performance.

- **Computational cost.** Each IF is extra work, even if the rest of the algorithm is simple.
- **Nested conditions.** IF inside IF often means a brute-force approach. It hurts both readability and speed.
- **Short-circuiting** (the optimisation goal). Stop evaluating as soon as the result is known. In a logical AND, if the first part is False the second part is skipped. In a logical OR, if the first part is True the second part is skipped.

WORKED EXAMPLE

Worked example: cheap test first. You only divide if the denominator is not zero. Place the cheap, safe test on the left so Python never evaluates the dangerous part.

```
x = 0
y = 10
if x != 0 and (y / x) > 2:
    print("ratio is large")
else:
    print("skipped safely")
```

OUTPUT

skipped safely

$x \neq 0$ is False, so AND short-circuits and y / x is never attempted. If you wrote the division first, the program would crash. Same idea in the soldier algorithm: a False height test skips Step 6.

REMEMBER IT

Put the **cheapest / most likely False** test first in an AND, and the **most likely True** test first in an OR. That is short-circuiting used on purpose.

COMMON MISTAKE

Mistake: writing a chain of nested IFs when one compound condition would do (as in the velocity example in 2.2.2).

Fix: flatten with AND/OR, and rely on short-circuiting instead of extra layers.

اردو میں سمجھیں

ہر IF پروسیسر سے True/False پوچھتی ہے اس لیے زیادہ شاخیں کارکردگی گھٹاتی ہیں۔ اندرونی IF اکثر brute-force ہوتی ہے۔ شارٹ سرکٹ (Short-circuit) کا مطلب ہے جیسے ہی جواب معلوم ہو جانچ روک دی جائے، مثلاً AND میں پہلی شرط False ہو تو دوسری نہ چلے۔

CHECK YOURSELF

1. Why do extra IF statements reduce performance?
2. What does nested IF often indicate?
3. In $x \neq 0$ and $(y / x) > 2$ with $x = 0$, why is there no division-by-zero?

Answers: 1. Each condition must be evaluated True/False by the CPU. 2. A brute-force approach (and it reduces readability and speed). 3. AND short-circuits: the first part is False, so the division is never evaluated.

2.3.3 Number of Repetitions

LEARN IT

Repetitions (loops) are the most significant factor in algorithm efficiency. The **structure of the loop** decides how time grows with n .

Type of repetition	Conceptual impact	Efficiency grade
Linear loop	Processes each element exactly once. Time grows in proportion to size.	Good — $O(n)$
Logarithmic repetition	Reduces the problem size by half each iteration (binary search). Huge time savings.	Excellent — $O(\log n)$
Nested loops	For each element, process all other elements. Operations explode.	Poor — $O(n^2)$

WORKED EXAMPLE

Worked example: count the inner-body runs.

```
n = 5
linear = 0
for i in range(n):
    linear = linear + 1

nested = 0
for i in range(n):
    for j in range(n):
        nested = nested + 1

print(linear, nested)
```

OUTPUT

5 25

One loop: 5. Nested loops: $25 = 5^2$. For $n = 1,000$ that would be 1,000 versus 1,000,000.

REMEMBER IT

Find the **deepest loop**. One loop $\approx O(n)$. Loop inside a loop $\approx O(n^2)$. A loop that halves $n \approx O(\log n)$.

COMMON MISTAKE

Mistake: counting the word `for` and ignoring how the index moves.

Fix: a loop `while n > 1: n = n // 2` is logarithmic, even though it is still “a loop”.

اردو میں سمجھیں

دہرائی (لوپ) کارکردگی کا سب سے بڑا عامل ہے۔ ایک بار ہر عنصر پر چلنے والا لوپ $O(n)$ (اچھا)، ہر بار مسئلہ آدھا کرنے والا $O(\log n)$ (بہترین)، اور اندرونی لوپ $O(n^2)$ (خراب) ہے۔ اندرونی جسم کے چلنے کی گنتی سے یہ فرق صاف نظر آتا ہے۔

CHECK YOURSELF

1. What grade does the source give nested loops?
2. Why is binary search's loop logarithmic rather than linear?
3. If $n = 5$, how many times does a fully nested n -by- n pair of loops run?

Answers: 1. Poor — $O(n^2)$. 2. Each iteration halves the remaining problem, so the number of iterations is about $\log n$. 3. 25.

2.3.4 The Efficiency Evaluation Framework

LEARN IT

To evaluate an algorithm at a conceptual level, ask three questions:

1. **Does it scale?** If n grows 10 times, does time jump 10 times (linear) or 100 times (quadratic)? Quadratic scaling fails for large datasets.
2. **What is the bottleneck?** Focus on the most time-consuming section (usually the deepest loop) and try to improve that part.
3. **Is the cost worth it?** A theoretically faster $O(\log n)$ algorithm can be too complex to code and maintain. For a list of just 50 items, a simple $O(n)$ linear search is practically better because it is simple.

Does it scale?



Where is the bottleneck?



Is the extra complexity worth it?

WORKED EXAMPLE

Worked example: 50 names versus 1 million names.

Situation	Scale	Bottleneck	Worth the cost?	Choice
50 unsorted class names	Tiny n	A single linear scan	Binary search needs a sort first, which is extra work and extra code	Linear search $O(n)$
1,000,000 sorted account numbers, searched all day	$n \times 10$ compared with 100,000 is already huge; quadratic would be impossible	The search loop	Yes — binary search is standard and well understood	Binary search $O(\log n)$

REMEMBER IT

Framework mnemonic: **S-B-W** — **S**cale, **B**ottleneck, **W**orth it.

COMMON MISTAKE

Mistake: always choosing the theoretically fastest Big O.

Fix: question 3 exists for a reason. For $n = 50$, simple $O(n)$ beats a fragile $O(\log n)$ implementation.

اردو میں سمجھیں

کارکردگی کے تین سوال: کیا الگورتھم بڑے n پر چل سکتا ہے (Scale)؟ سب سے سست جگہ کون سی ہے (Bottleneck)، عموماً گہرا لوپ؟ کیا تیز تر طریقے کی پیچیدگی اس چھوٹے فائدے کے لائق ہے؟ پچاس ناموں کے لیے سادہ $O(n)$ تلاش عملی طور پر بہتر ہو سکتی ہے۔

CHECK YOURSELF

1. If n grows $10\times$, how much slower is a quadratic algorithm?
2. Where do you usually look for the bottleneck?
3. Why might linear search beat binary search for 50 items?

Answers: 1. About $100\times$ slower (10^2). 2. The most time-consuming section, usually the deepest loop. 3. Binary search is more complex to code and maintain, and for $n = 50$ the simplicity of $O(n)$ is practically better.

2.4 Refinements to improve clarity and efficiency

LEARN IT

Improving an algorithm means reviewing it step by step: simplifying logic, removing redundant work, and picking smart data structures. A clear algorithm is easier to follow; an efficient one wastes fewer steps.

2.4.1 Refinements for clarity

1. **Use meaningful names.** Replace vague variables with descriptive ones. Bad: `a = b / c`. Good: `average = totalSum / totalItems`.
2. **Break into modules.** Divide a massive block into functions, for example separating `CalculateSum()` from the printing logic.
3. **Improve structure and indentation.** Packed loops and IF statements confuse the reader. Spacing reduces mental effort.
4. **Add comments.** Keep them short. Explain **why** something is done, not just what is happening, especially for tricky logic.

2.4.2 Refinements for efficiency

1. **Reduce unnecessary steps.** In Bubble Sort, after the first pass the largest value has settled at the end. Ignore that sorted tail on the next pass by shrinking the inner loop, and you save comparisons.
2. **Use better algorithms.** Searching 1 million **sorted** records with Binary Search instead of Linear Search changes the time from seconds to milliseconds.
3. **Avoid re-computing values** (memoization / caching). If a value never changes, compute it once outside the loop, store it, and reuse it.
4. **Use appropriate data structures.** Direct access $\rightarrow$ array. Hierarchical data $\rightarrow$ tree. Do not force one structure everywhere.

WORKED EXAMPLE

Worked example 1 — shrinking Bubble Sort's inner loop ($n = 5$). A naive inner loop always runs $n-1 = 4$ times, for 4 passes: 16 comparisons. The refined inner loop runs 4, then 3, then 2, then 1: 10 comparisons.

```
n = 5
naive = 0
for i in range(n - 1):
    for j in range(n - 1):
        naive = naive + 1

refined = 0
for i in range(n - 1):
    for j in range(n - 1 - i):
        refined = refined + 1

print(naive, refined)
```

OUTPUT

```
16 10
```

Worked example 2 — do not re-compute a constant inside the loop.

```
A = 3
B = 4
N = 5

# BAD: C is recalculated every time
for i in range(1, N + 1):
    C = A + B
    print(C * i)
```

OUTPUT

```
7
14
21
28
35
```

```
A = 3
B = 4
N = 5

# GOOD: compute C once
C = A + B
for i in range(1, N + 1):
    print(C * i)
```

OUTPUT

```
7
14
21
28
35
```

Same output, fewer additions. That is caching: compute once, reuse N times.

REMEMBER IT

Clarity: **N-M-I-C** (Names, Modules, Indentation, Comments). Efficiency: **S-A-C-D** (Shrink steps, Algorithm choice, Cache, Data structure).

COMMON MISTAKE

Mistake: "optimising" by renaming variables and thinking that is an efficiency refinement.

Fix: meaningful names improve **clarity**. Efficiency refinements change the **amount of work** (fewer loop iterations, a better algorithm, no re-compute, a better structure).

اردو میں سمجھیں

وضاحت بہتر کرنے کے لیے معنی خیز نام، ماڈیول، درست indentation اور مختصر تبصرے استعمال کریں۔ کارکردگی کے لیے فضول قدم ہٹائیں (بیل سارٹ میں ترتیب شدہ دم نظر انداز کریں)، بہتر الگورتھم چنیں (لینئر کی جگہ ہائری سرچ)، لوپ کے اندر وہی حساب بار بار نہ کریں (Caching/Memoization)، اور مناسب ڈیٹا سٹرکچر رکھیں۔

CHECK YOURSELF

1. Rewrite `a = b / c` with meaningful names for an average.
2. After Bubble Sort's first pass, what should the next pass ignore?
3. In the A+B example, where should `C = A + B` be written?

Answers: 1. `average = totalSum / totalItems`. 2. The already-sorted largest value at the end (shrink the inner loop). 3. Once, outside the loop, then reuse C inside.

2.5 Concept of Data Structure ★ Golden

LEARN IT

A **data structure** is a systematic way of **organising, storing and managing** data so that an algorithm can access, modify and process it efficiently.

Real-life analogy. Books on a shelf are arranged in a set order, so you can find a title quickly. Files in folders are grouped on purpose. Data structures do the same job inside computer memory.

1. Linear data structures

Elements stand in a strict sequence, one after another. Each element (except the first and last) has exactly **one predecessor** and **one successor**.

Examples: Array, Linked List, Stack, Queue.

Linear (sequence)

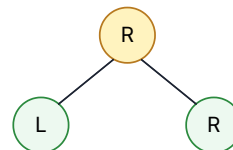


2. Non-linear data structures

Elements are connected in a hierarchy or a network. One element can connect to **several** others at the same time.

Examples: Tree, Graph.

Non-linear (branch / net)



Linear: a line of neighbours. Non-linear: a node with more than one connection.

WORKED EXAMPLE

Worked example: classify each everyday picture.

1. Students standing in a single assembly line → **linear** (each has one in front and one behind).
2. A family tree of grandparents, parents and children → **non-linear** (one parent, several children).
3. Roads linking Karachi, Hyderabad, Sukkur and Larkana in many ways → **non-linear** (a network).

REMEMBER IT

Ask: "Does each item have at most one next item?" Yes → linear. "Can one item link to many, or can links form a net?" → non-linear.

COMMON MISTAKE

Mistake: calling a tree "linear" because you can walk it from top to bottom.

Fix: walking is traversal. The **shape** is hierarchical: one parent, many children. That is non-linear.

اردو میں سمجھیں

ڈیٹا سٹرکچر ڈیٹا کو منظم طریقے سے رکھنے کا نظام ہے تاکہ الگورتھم اسے جلدی استعمال کر سکے۔ لکیری ڈھانچوں میں عناصر قطار میں ہوتے ہیں (ایرے، لنکڈ لسٹ، سٹیک، کیو)۔ غیر لکیری میں ایک عنصر کئی سے جڑ سکتا ہے (ٹری، گراف)۔ الماری کی کتابیں اور فولڈرز کی مثال یاد رکھیں۔

EXAM TIP

Define data structure in full (organise, store, manage; efficient access and processing). Then split linear vs non-linear with the "one predecessor and one successor" test, and list the four linear and two non-linear examples. The shelf / folder analogy earns a mark. MCQs: "Which is non-linear?" → Tree or Graph.

CHECK YOURSELF

1. Define data structure.
2. Name four linear and two non-linear examples.
3. What is true of each middle element in a linear structure?

Answers: 1. A systematic way of organising, storing and managing data for efficient access, modification and processing. 2. Linear: array, linked list, stack, queue. Non-linear: tree, graph. 3. It has exactly one predecessor and one successor.

2.5.1 Linear Data Structures: Array

LEARN IT

An **array** holds a collection of **similar items** (the same data type) placed next to each other in **contiguous memory** (one continuous block).

Each item has a position number called an **index**, usually starting from **0**. Because the address of any cell can be calculated from the index, you can jump straight to it. That is **direct access** (also called random access), and it is $O(1)$.

Array name: A

23	76	12	90	34	68
0	1	2	3	4	5

A[3] = 90 (index starts at 0)

Six integers in contiguous cells. Direct access: A[3] jumps to 90 without walking 23, 76 and 12.

WORKED EXAMPLE

Worked example: read by index.

```
A = [23, 76, 12, 90, 34, 68]
print(A[3])
print(A[0])
```

OUTPUT

```
90
23
```

REMEMBER IT

Index 0 is the first shelf on the row. The third item that humans call "number 4" is index 3. Formula in your head: **position from the left, minus one.**

COMMON MISTAKE

Limitation. Arrays are fast to read by index but **inflexible** to grow in the middle. Inserting a new value between 12 and 90 means shifting 90, 34 and 68 one cell to the right. That shift is $O(n)$, so arrays are a poor fit when you insert and delete in the middle all the time.

اردو میں سمجھیں

ایرے ایک جیسے عناصر کو میموری کے لگاتار بلاک میں رکھتا ہے۔ ہر عنصر کا انڈیکس (عموماً 0 سے) ہوتا ہے جس سے براہ راست رسائی میں مل جاتی ہے، جیسے $A[3] = 90$ ۔ درمیان میں نیا عنصر ڈالنے کے لیے دائیں طرف شفٹ کرنا پڑتا ہے جو سست ہے۔ $O(1)$

CHECK YOURSELF

1. What does "contiguous memory" mean?
2. In the diagram, what is A[2]?
3. Why is inserting in the middle of an array inefficient?

Answers: 1. The items sit in one continuous block, next to each other. 2. 12. 3. Later elements must all be shifted right to make room.

Linear Data Structures: Linked List

LEARN IT

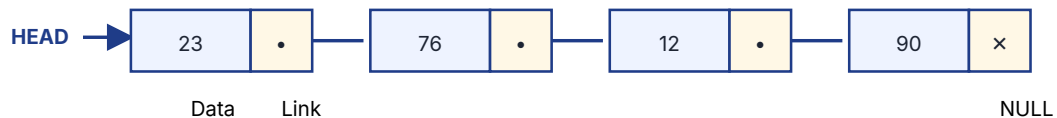
A **linked list** stores data as a chain. Unlike an array, the items are **not** one continuous block. Each separate piece is a **node** sitting somewhere in memory.

Each node has two parts:

1. the actual **data**;
2. a **link** (pointer) holding the memory address of the **next** node.

The first node is pointed to by **HEAD**. The last node's link is **NULL** (nowhere). To find an item you start at HEAD and follow links one by one — **sequential access**, $O(n)$.

Real-life example: browser history. Clicking Back follows the pointer to the previous page-node.



Follow HEAD → 23 → 76 → 12 → 90 → NULL

Four nodes. The last link is NULL, which ends the chain.

WORKED EXAMPLE

Worked example: walk from HEAD to print every data field.

```
nodes = [23, 76, 12, 90]
for data in nodes:
    print(data, end=" → ")
print("NULL")
```

OUTPUT

23 → 76 → 12 → 90 → NULL

There is no `nodes[2]` jump in a real linked list. The loop above models following each link once. Inserting a new node between 76 and 12 only means changing two pointers — you do not shift the rest of a big block, which is why lists beat arrays for frequent insert/delete.

REMEMBER IT

Array = row of lockers with numbers on the doors. Linked list = treasure hunt: each clue (link) tells you where the next box is.

COMMON MISTAKE

Mistake: writing `list[3]` and calling it linked-list access.

Fix: a true linked list has no index. Python's `list` type is an array-like structure. In this chapter, "linked list" means HEAD-and-pointers.

اردو میں سمجھیں

لنکڈ لسٹ زنجیر کی طرح نوڈز پر مشتمل ہے جو میموری میں بکھرے ہو سکتے ہیں۔ ہر نوڈ میں ڈیٹا اور اگلے نوڈ کا پتہ (Link) ہوتا ہے۔ پہلا نوڈ HEAD سے ملتا ہے، آخری کا لنک NULL ہوتا ہے۔ براؤزر کا Back بٹن پچھلے نوڈ کے پوائنٹر پر چلتا ہے۔ رسائی ترتیب وار ہے، انڈیکس سے چھلانگ نہیں۔

CHECK YOURSELF

1. Name the two parts of a node.
2. What does the last node's link store?
3. Why is browser history a fair analogy?

Answers: 1. Data, and a link/pointer to the next node. 2. NULL (no next node). 3. Back follows the pointer to the previous page-node instead of jumping by index.

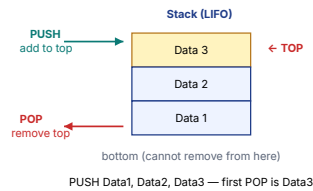
Linear Data Structures: Stack ★ Golden

LEARN IT

A **stack** is a linear data structure that works on the **LIFO** principle: **Last-In, First-Out**. The last element added is the first one removed. You cannot pull an item from the bottom or the middle.

- **PUSH** — add an item to the **top**.
- **POP** — remove the item from the **top**.

A **TOP** pointer always names the current top item. **Real-life example:** Undo in a text editor. Every action is piled on the stack; Undo POPs the most recent action first.



Three PUSH operations. The TOP is Data 3, so the first POP returns Data 3.

WORKED EXAMPLE

Worked example: PUSH / POP trace. Start with an empty stack. Then PUSH Data1, PUSH Data2, PUSH Data3, POP, POP.

Step	Operation	Stack (bottom → top)	TOP	Popped value
0	(empty)	[]	none	
1	PUSH Data1	[Data1]	Data1	
2	PUSH Data2	[Data1, Data2]	Data2	
3	PUSH Data3	[Data1, Data2, Data3]	Data3	
4	POP	[Data1, Data2]	Data2	Data3
5	POP	[Data1]	Data1	Data2

```
stack = []
stack.append("Data1")
stack.append("Data2")
stack.append("Data3")
print("after 3 PUSHes:", stack)
print("POP:", stack.pop())
print("POP:", stack.pop())
print("left:", stack)
```

```
OUTPUT
after 3 PUSHes: ['Data1', 'Data2', 'Data3']
POP: Data3
POP: Data2
left: ['Data1']
```

Python's `list.append` is PUSH and `list.pop()` (no index) is POP from the top.

REMEMBER IT

LIFO = Last In First Out = "last plate washed is first plate used" on a canteen pile. Undo, Redo, Back button and function calls are all stacks.

COMMON MISTAKE

Mistake: POPing from the bottom, or mixing LIFO with FIFO.

Fix: stack = plates (top only). Queue = ticket line (front leaves, rear joins). If the first POP is Data1, you drew a queue, not a stack.

اردو میں سمجھیں

سٹیک LIFO پر چلتا ہے: جو سب سے بعد میں آیا وہ سب سے پہلے نکلے گا۔ اوپر رکھنا PUSH، اوپر سے ہٹانا POP، اور TOP پوائنٹر موجودہ سب سے اوپر عنصر بتاتا ہے۔ انڈو (Undo) کی مثال: ہر عمل ڈھیر پر پڑتا ہے، انڈو سب سے تازہ عمل ہٹاتا ہے۔

EXAM TIP

Define stack + LIFO, define PUSH and POP, mention TOP, give the Undo example, and draw the vertical boxes with TOP labelled. Trace questions: "Show the stack after PUSH A, PUSH B, POP, PUSH C." Work left to right; the first POP must be B. Never remove from the middle.

CHECK YOURSELF

1. Expand LIFO.
2. After PUSH Data1, Data2, Data3, what does the first POP return?
3. Why is Undo modelled as a stack?

Answers: 1. Last-In, First-Out. 2. Data3. 3. Each action is PUSHed; Undo POPs the most recent action first.

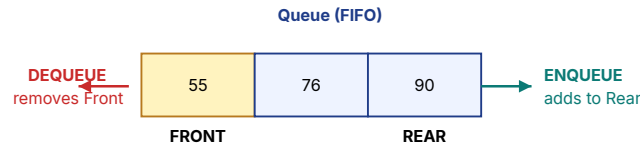
Linear Data Structures: Queue ★ Golden

LEARN IT

A **queue** organises data on the **FIFO** principle: **First-In, First-Out**. Whoever arrives first is served first. Order truly matters.

- **ENQUEUE** — add an element at the **Rear** (back).
- **DEQUEUE** — remove an element from the **Front**.

Real-life example: several documents sent to a printer. The printer prints the first document received, then the next.



Next ENQUEUE joins after 90. Next DEQUEUE removes 55.

Three jobs waiting. New jobs join at the rear; service always takes the front.

WORKED EXAMPLE

Worked example: ENQUEUE / DEQUEUE trace. Start empty. ENQUEUE 55, 76, 90, then ENQUEUE D, then DEQUEUE.

Step	Operation	Queue (Front → Rear)	Front	Rear	Removed
0	(empty)	[]	—	—	
1	ENQUEUE 55	[55]	55	55	
2	ENQUEUE 76	[55, 76]	55	76	
3	ENQUEUE 90	[55, 76, 90]	55	90	
4	ENQUEUE D	[55, 76, 90, D]	55	D	
5	DEQUEUE	[76, 90, D]	76	D	55

```
from collections import deque
q = deque()
q.append(55)
q.append(76)
q.append(90)
q.append("D")
print("after ENQUEUE D:", list(q))
print("DEQUEUE:", q.popleft())
print("left:", list(q))
```

OUTPUT
after ENQUEUE D: [55, 76, 90, 'D']
DEQUEUE: 55
left: [76, 90, 'D']

append is ENQUEUE at the rear; popLeft is DEQUEUE from the front. The first DEQUEUE is 55 (the oldest), never D.

REMEMBER IT

FIFO = First In First Out = a canteen line. Front = the person being served. Rear = the person who just joined.

COMMON MISTAKE

Mistake: DEQUEUEing from the rear, or ENQUEUEing at the front.

Fix: new items only at Rear; removal only at Front. If your first removal is D, you treated the queue as a stack.

اردو میں سمجھیں

کیو FIFO پر چلتی ہے: جو پہلے آیا وہ پہلے نکلے۔ پیچھے (Rear) شامل کرنا ENQUEUE، آگے (Front) سے بنانا DEQUEUE ہے۔ پرنٹر کی قطار: جو فائل پہلے بھیجی گئی وہ پہلے چھپے گی۔ پہلی DEQUEUE ہمیشہ سب سے پرانا عنصر نکالتی ہے۔

EXAM TIP

Define queue + FIFO, define ENQUEUE (rear) and DEQUEUE (front), give the printer example, draw the labelled Front/Rear diagram, and trace a short sequence. Examiners mix stack and queue on the same paper: if they say "first come, first served" write Queue; if they say "Undo" write Stack.

CHECK YOURSELF

1. Expand FIFO. Where is an item added, and where is it removed?
2. After ENQUEUE 55, 76, 90, what does DEQUEUE return?
3. Why is a printer a queue, not a stack?

Answers: 1. First-In, First-Out; added at Rear, removed from Front. 2. 55. 3. Documents must print in the order they arrived (first received, first printed), not last-received first.

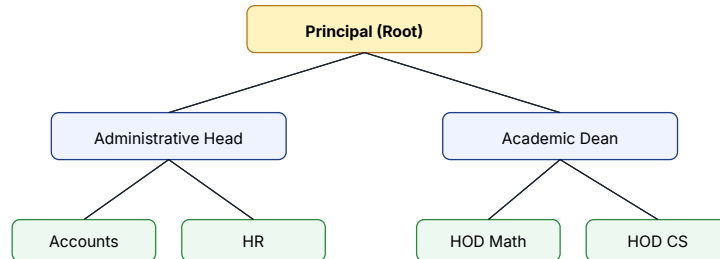
2.5.2 Non-Linear Data Structures: Tree

LEARN IT

A **tree** organises data in a **hierarchy**. Everything branches out from a single starting point called the **root**.

It creates a strict **one-to-many (parent-child)** relationship. That is an excellent way to show how roles flow in an organisation.

Real-life examples: a family tree (grandparents → parents → children) and folders inside folders on a drive.



College tree: one root (Principal), then parent-child branches. No child has two parents in this model.

WORKED EXAMPLE

Worked example: name the relationships.

- Root: Principal.
- Children of the root: Administrative Head, Academic Dean.
- Children of Academic Dean: HOD Math, HOD CS.
- HOD CS has one parent (Academic Dean), not two — that is the tree rule. If HOD CS also reported sideways to Accounts as an equal link, the picture would become a **graph**.

REMEMBER IT

Tree = family / folders / organogram. One root. Branches down. Parent-child, not a web of equals.

COMMON MISTAKE

Mistake: drawing a tree with two roots, or with an arrow going back up to the Principal from HOD CS (a cycle).

Fix: one root, links only parent → child, no loops. Cycles belong to graphs.

اردو میں سمجھیں

ٹری ڈیٹا کو درجہ بندی (Hierarchy) میں رکھتی ہے جو ایک جڑ (Root) سے پھوٹتی ہے۔ تعلق والدین-بچہ (ایک سے کئی) ہوتا ہے، جیسے پرنسپل → ڈین → ایچ او ڈی، خاندانی شجرہ، یا فولڈر کے اندر فولڈر۔ دو جڑیں یا چکر ٹری کو گراف بنا دیتے ہیں۔

CHECK YOURSELF

1. What is the starting node of a tree called?
2. What kind of relationship does a tree model?
3. Give two real-life tree examples from the source.

Answers: 1. The root. 2. One-to-many parent-child. 3. A family tree, and folders inside folders (or a college organogram).

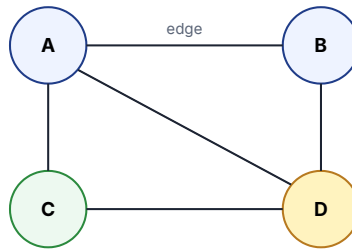
Non-Linear Data Structures: Graph

LEARN IT

A **graph** organises data when things connect in a flexible, complex way. Unlike a tree, there is **no strict hierarchy**: any node can connect to any other node.

- **Nodes (vertices)** — the items (cities, people).
- **Edges** — the connections (roads, friendships).

Examples: Facebook friends; GPS mapping (the fastest route between two cities).



Four vertices A, B, C, D. Edges can form a cycle (A–B–D–C–A). A tree cannot do that.

WORKED EXAMPLE

Worked example: Karachi – Hyderabad – Sukkur – Larkana. Cities are vertices. Roads are edges. You can travel Karachi → Hyderabad → Sukkur or Karachi → Larkana → Sukkur. There is no single “root city”. That is a graph, and GPS uses it to pick a fast route.

REMEMBER IT

Tree $\subset$ Graph in shape-talk: a tree is a special graph with one root and no cycles. If you need cycles or many-to-many links, say **graph**.

COMMON MISTAKE

Mistake: using a tree for GPS because “Hyderabad comes under Sindh”.

Fix: administration can be a tree; **roads** are a graph because you can loop and take alternative paths.

اردو میں سمجھیں

گراف لچکدار روابط کے لیے ہے: کوئی سخت درجہ بندی نہیں، کوئی بھی نوڈ کسی سے جڑ سکتا ہے۔ نوڈز (Vertices) چیزیں ہیں، کنارے (Edges) تعلق۔ فیس بک دوست اور GPS نقشہ کلاسیکی مثالیں ہیں۔ چکر (Cycle) اور کئی راستے گراف کو ٹری سے الگ کرتے ہیں۔

CHECK YOURSELF

1. What are the two parts of a graph?
2. Give two source examples of graphs.
3. State one difference between a tree and a graph.

Answers: 1. Nodes/vertices and edges. 2. Social networks (Facebook friends) and GPS mapping. 3. A tree has a hierarchy and one root; a graph has no strict hierarchy and any node may connect to any other (cycles allowed).

2.5.3 Operations on Data Structures

LEARN IT

Whatever the shape, you keep meeting the same five operations:

Operation	Meaning	Quick picture
Insertion	Adding a new item into the structure.	PUSH on a stack; ENQUEUE on a queue; putting 40 into an array.
Deletion	Removing an item that is no longer needed.	POP; DEQUEUE; deleting a node from a list.
Traversal	Going through each item one by one.	Walking HEAD → ... → NULL; visiting every tree node.
Searching	Finding a specific item.	Linear search in an array; following links until 87 is found.
Sorting	Arranging items in a proper order (ascending or descending).	Bubble Sort on an array of marks.

WORKED EXAMPLE

Worked example: name the operation.

1. "Print every song from the start of the playlist." → **Traversal**.
2. "Add a print job at the rear." → **Insertion** (ENQUEUE).
3. "Undo the last typing action." → **Deletion** (POP).
4. "Is roll number 87 in the list?" → **Searching**.
5. "Arrange heights from shortest to tallest." → **Sorting**.

REMEMBER IT

Memorise **I-D-T-S-S**: Insert, Delete, Traverse, Search, Sort.

COMMON MISTAKE

Mistake: calling a PUSH "sorting" because the stack looks tidy.

Fix: PUSH is insertion. Sorting means putting values in ascending or descending order.

اردو میں سمجھیں

برڈیٹا سٹرکچر پر پانچ عمل ہوتے ہیں: داخل کرنا (Insertion)، حذف کرنا (Deletion)، ایک ایک کر کے گزرنا (Traversal)، تلاش (Searching) اور ترتیب (Sorting، صعودی یا نزولی)۔

CHECK YOURSELF

1. List the five operations.
2. Which operation is ENQUEUE?
3. Traversal vs searching — one difference?

Answers: 1. Insertion, deletion, traversal, searching, sorting. 2. Insertion (at the rear of a queue). 3. Traversal visits every item; searching stops when the target is found (or the structure ends).

2.6 Identify data structures in problem scenarios

LEARN IT

To pick a structure, first see how the data is organised. If each item has a clear “before and after”, it is **linear**. If it branches or connects to many items, it is **non-linear**.

2.6.1 Linear scenarios

1. **Storing student grades for a class → Array.** You usually know how many students there are, and all grades are the same type. Indices give rapid lookup.
2. **Music playlist / browser history → Linked list.** Every song knows what comes next. When one finishes, follow the pointer. Frequent insert/delete is cheap.
3. **Undo or Redo → Stack (LIFO).** Each typing or formatting action is PUSHed. Undo POPs the most recent action.
4. **Printer task scheduling → Queue (FIFO).** Documents print in the exact order they were received. First-come, first-served.

2.6.2 Non-linear scenarios

1. **Organisational chart of a college → Tree.** Principal is the root, then deans, then HODs, then teachers. Clear parent-child links.
2. **GPS navigation and maps → Graph.** Cities are nodes, roads are edges. No strict top-down hierarchy. Several routes can reach the same destination.

If the data...	Then it is...	Recommended structure
Follows “next” or “previous” logic	Linear	Array, Linked list
Follows “parent” and “child” logic	Non-linear	Tree
Follows “first-come, first-served” logic	Linear	Queue
Follows “connections / relationships” logic	Non-linear	Graph
Needs to be accessed in the reverse order of entry	Linear	Stack

WORKED EXAMPLE

Worked example: match five school problems.

Scenario	Structure	Why (one line)
Store 40 practical marks you already counted	Array	Fixed size, same type, need A[i] lookup.
A playlist where the teacher inserts a nasheed between two songs	Linked list	Insert by moving two pointers, no big shift.
Ctrl+Z after formatting a heading	Stack	Last action must come off first.
Students lining up for the photocopier	Queue	First in line is served first.
Friend links in a class WhatsApp “everyone knows everyone”	Graph	Any student can connect to any other.

REMEMBER IT

Decision order: reverse-of-entry? → stack. First-come? → queue. Parent-child? → tree. Many-to-many routes? → graph. Known list with indexes? → array. Lots of insert/delete in a sequence? → linked list.

COMMON MISTAKE

Mistake: choosing an array for Undo because “actions are stored in a list”.

Fix: the **access rule** matters more than “it is a list”. Undo is reverse order of entry → stack.

اردو میں سمجھیں

اگر ہر عنصر کا واضح اگلا/پچھلا ہے تو ڈھانچہ لکیری ہے؛ اگر شاخیں یا کئی روابط ہیں تو غیر لکیری۔ نمبر: ایرے؛ پلے لسٹ/ہسٹری: لنکڈ لسٹ؛ انڈو: سٹیک؛ پرنٹر: کیو؛ کالج کا چارٹ: ٹری؛ GPS: گراف۔ فیصلے کی جدول یاد کر لیں۔

CHECK YOURSELF

1. Which structure stores a class's grades, and why?
2. Printer jobs: stack or queue? Why?
3. “Needs reverse order of entry” maps to which structure?

Answers: 1. Array — known size, same type, fast index lookup. 2. Queue, because first-come, first-served. 3. Stack.

2.7 Trace data retrieval in Array, List and Queue

LEARN IT

Tracing data retrieval means following the step-by-step path of how an algorithm finds or accesses data. Array, linked list and queue are all linear, but the **effort** to find an item is very different because of how they are organised inside.

2.7.1 Retrieval in an array

Every element has a fixed index from 0 to n-1 pointing at a spot in memory. If the algorithm already knows the index, it does not search: it **jumps** (direct access, **O(1)**).
Array = [10, 50, 20, 70, 90, 30]. Direct access: Array[2] = 20.

Loop iteration	Index	Array access	Value
1	0	Array[0]	10
2	1	Array[1]	50
3	2	Array[2]	20

```
Array = [10, 50, 20, 70, 90, 30]
print("Direct Array[2] =", Array[2])
for i in range(3):
    print(i + 1, i, "Array[{}]", Array[i])
```

```
OUTPUT
Direct Array[2] = 20
1 0 Array[0] 10
2 1 Array[1] 50
3 2 Array[2] 20
```

2.7.2 Retrieval in a linked list

Linked lists have **no index**. The only way is to start at HEAD and follow links one node at a time (sequential access, **O(n)**).
List: [12] → [98] → [23] → [87] → [56] → NULL. Search key: 87.

Step	Node data	Compare with 87	Action
1	12	12 != 87	Move next
2	98	98 != 87	Move next
3	23	23 != 87	Move next
4	87	87 == 87	FOUND — output True

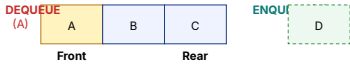
```
nodes = [12, 98, 23, 87, 56]
key = 87
found = False
for i, value in enumerate(nodes, start=1):
    print(f"Step {i}: Check Node {i} ({value}).")
    if value == key:
        found = True
        print("FOUND! Output: True")
        break
    print("Not equal. Move Next.")
print("Result:", found)
```

```
OUTPUT
Step 1: Check Node 1 (12).
Not equal. Move Next.
Step 2: Check Node 2 (98).
Not equal. Move Next.
Step 3: Check Node 3 (23).
Not equal. Move Next.
Step 4: Check Node 4 (87).
FOUND! Output: True
Result: True
```

Key 87 sits at the fourth node, so you paid four checks. An array that already knew index 3 would have paid one jump.

2.7.3 Retrieval in a queue

A queue is FIFO. You **cannot** retrieve from the middle or the rear. The oldest item at the Front is always DEQUEUED first.



You cannot pull B until A has left.

Queue A, B, C. ENQUEUE D joins the rear. DEQUEUE must remove A, not C or B.

```
from collections import deque
q = deque(["A", "B", "C"])
q.append("D")
print("after ENQUEUE D:", list(q))
print("DEQUEUE:", q.popleft())
print("now:", list(q))
```

```
OUTPUT
after ENQUEUE D: ['A', 'B', 'C', 'D']
DEQUEUE: A
now: ['B', 'C', 'D']
```

WORKED EXAMPLE

Structure	Target	Path	Can you jump?
Array [10, 50, 20, ...]	20 at index 2	Array[2] in one step, or scan 10, 50, 20	Yes, if the index is known
Linked list 12→98→23→87→...	87	Must visit 12, 98, 23, then 87	No
Queue A-B-C	A (the only legal retrieval)	DEQUEUE Front	No — never the middle

REMEMBER IT

Array jumps, list walks, queue waits. If the exam asks "why is array retrieval different?", write **direct access by index** versus **sequential links** versus **Front only**.

COMMON MISTAKE

Mistake: DEQUEUEing C because "C is the interesting letter", or reading linked-list node 4 as list[3] without walking.
Fix: show every hop in the table. Marks are for the path, not only for FOUND.

اردو میں سمجھیں

اگر انڈیکس معلوم ہو تو سیدھی چھلانگ لگتی ہے، جیسے Array[2] = 20. لنکڈ لسٹ میں HEAD سے ایک ایک نوڈ دیکھنا پڑتا ہے، 87 چوتھے نوڈ پر ملا۔ کیوں صرف Front نکلتا ہے، درمیان سے نہیں: A, B, C میں DEQUEUE ہمیشہ A ہے۔

CHECK YOURSELF

1. What is Array[2] in [10, 50, 20, 70, 90, 30]?
2. How many nodes are checked before 87 is found in 12→98→23→87→56?
3. In queue A, B, C, can you retrieve B without removing A?

Answers: 1. 20. 2. Four (12, 98, 23, 87). 3. No — a queue only DEQUEUES the Front, so A must leave first.

Career Opportunities in Algorithms

LEARN IT

The skills in this chapter — proving correctness, writing clear modules, judging Big O, and choosing data structures — are the daily tools of several computing careers.

Career	What the person does
Software Engineer	Builds and maintains software that relies on data structures and efficient algorithms to solve logic problems.
Backend Developer	Builds server-side logic where choosing the right structure (array, queue) directly affects web performance.
Algorithm Engineer	Specialises in optimising algorithms for search engines, finance or logistics.
Systems Analyst	Evaluates and improves the efficiency and clarity of existing computer systems in an organisation.
Competitive Programmer	Solves complex algorithmic problems under time limits, a common path into top tech roles.

WORKED EXAMPLE

Worked example: match the task.

1. "Replace an $O(n^2)$ report with an $O(n \log n)$ sort so the payroll finishes overnight." → **Algorithm Engineer** (or a performance-minded Software Engineer).
2. "Put incoming HTTP requests into a FIFO buffer so the first visitor is served first." → **Backend Developer** (queue).
3. "Read a college's old fee software, find the nested-IF bottleneck, and recommend modules." → **Systems Analyst**.

REMEMBER IT

Every career in the table uses the same three tests you learned: **correct, clear, efficient** — plus the right data structure.

COMMON MISTAKE

Mistake: thinking only "Algorithm Engineer" needs Big O.

Fix: a backend developer who puts a nested loop on every page request will take a site down. Efficiency is a shared skill.

اردو میں سمجھیں

اس باب کی مہارتیں سافٹ ویئر انجینئر، بیک اینڈ ڈیولپر، الگورتھم انجینئر، سسٹمز اینالسٹ اور کمپیٹیو پروگرامر کے کام میں روز استعمال ہوتی ہیں: درست منطق، صاف ماڈیول، بگ او اور درست ڈیٹا سٹرکچر۔

CHECK YOURSELF

1. Which career specialises in search-engine or logistics algorithms?
2. Why does a backend developer care about queues and arrays?

Answers: 1. Algorithm Engineer. 2. The choice of structure (array, queue) directly affects web performance.

Computational Thinking & Algorithms

24 single-page lessons · 6 Golden · read, close, say aloud, then sit the exam

Chapter in one look

- **Correctness:** output matches expected results for valid inputs. Test with a trace table (manual dry run) or stepwise reasoning (logic verification).
- A **trace table** is micro-level: line, variables, I/O, conditions. Best for small loops. The soldier heights [1.91, 2.01, 1.87, 1.57, 1.48] print Total = 3.
- **Stepwise reasoning** is macro-level: valid start, all cases, correct conditions, loop stays correct. Sum 1..N starts at 0; N = 0 stays 0; N = 5 becomes 15.
- **Clarity** (read, understand, modify) often beats a tiny speed gain. It has two parts: modularity and readability.
- **Modularity:** SRP (one job), loose coupling, abstraction (call Sort without knowing Bubble vs Selection). Split CalculateSum and FindAverage.
- **Readability:** names (`velocity = distance / time`), self-documentation, simple flow, consistent style.
- **Efficiency** = time + space. Drivers: number of steps (Big O), conditions, repetitions.
- **Big O:** $O(1)$ constant (arr[0]), $O(\log n)$ binary search, $O(n)$ linear search, $O(n \log n)$ Merge Sort, $O(n^2)$ nested loops / Bubble Sort. Quadratic fails as n grows.
- **Framework:** Does it scale? What is the bottleneck (deepest loop)? Is the extra complexity worth it? For n = 50, simple $O(n)$ can beat fragile $O(\log n)$.
- **Refinements:** meaningful names, modules, indentation, why-comments; shrink Bubble Sort's inner loop; better algorithms; cache constants; pick the right structure.
- **Data structure:** systematic organising of data. Linear (one after another): array, linked list, stack, queue. Non-linear: tree, graph.
- **Array:** contiguous, index from 0, direct access $O(1)$, shifting on mid-insert. **Linked list:** nodes of data+link, HEAD to NULL, sequential access.
- **Stack LIFO:** PUSH/POP at TOP (Undo). **Queue FIFO:** ENQUEUE rear, DEQUEUE front (printer).
- **Tree:** one root, parent-child (college chart). **Graph:** vertices + edges, no strict hierarchy (GPS, friends).
- Five operations: insert, delete, traverse, search, sort. Choose by next/previous, parent/child, FIFO, relationships, or reverse-of-entry.
- **Retrieval:** array can jump to an index; a list walks from HEAD; a queue only removes the Front.

Key terms

Term	Meaning	اردو
Correctness	The algorithm's output matches the expected results for valid inputs.	درستگی
Trace table	A paper dry-run table of steps, variables, I/O and conditions.	ٹریس ٹیبل
Dry run	Executing an algorithm by hand, line by line.	دستی اجرا
Stepwise reasoning	Checking each logic block's state; also called logic verification.	مرحلہ وار استدلال
Clarity	How easy an algorithm is to read, understand and modify.	وضاحت
Modularity	Breaking work into independent functions, classes or modules.	ماڈیولریتی
Single Responsibility Principle	Each module should do one specific task.	واحد ذمہ داری کا اصول
Coupling	How tightly modules depend on each other; loose coupling is the goal.	ماڈیولز کی جڑت
Abstraction	Hiding implementation details so a caller need not know the inner method.	تجريد
Readability	How easily another person understands the logic without running it.	پڑھائی پن
Efficiency	How well an algorithm uses time (speed) and space (memory).	کارکردگی
Big O notation	A description of how running time or memory grows as n grows.	بگ او نوٹیشن

Computational Thinking & Algorithms

75 marks · 1 hour 15 minutes · no notes · no phones

CHAPTER EXAM RULES

Instructions. Sit this exam after Lesson 24 and the recap. **Section A** — all 15 MCQs (1 mark). **Section B** — all 10 short questions (3 marks). **Section C** — attempt any **three** long questions (10 marks). At least one Section C answer must be a ★ Golden topic. Write in your notebook. Do not open the key until time is up.

Marks. $15 + 30 + 30 = 75$. A 5-mark / 10-mark answer needs Define · Explain · Example · Diagram · Working.

سبق اور خلاصے کے بعد یہ امتحان بغیر نوٹس کے حل کریں۔ وقت پورا ہونے پر کلید کھولیں۔

Section A — Multiple choice (15 × 1 = 15)

1. An algorithm is correct when:
 - a. it compiles with no syntax error
 - b. its output matches the expected results for the given input
 - c. it uses Big O notation
 - d. it has no comments
2. A trace table is best described as:
 - a. a paper execution that tracks variables line by line
 - b. a diagram of a tree
 - c. a type of graph edge
 - d. an $O(n^2)$ nested loop
3. Stepwise reasoning mainly checks:
 - a. printer ink levels
 - b. the colour of variable names
 - c. only the last printed line
 - d. the overall state of logic blocks
4. Which method is impractical if a loop runs 1,000 times?
 - a. Abstraction
 - b. A full trace table
 - c. Stepwise reasoning
 - d. Using a function
5. The Single Responsibility Principle says each module should:
 - a. focus on one specific task
 - b. print and save and calculate everything
 - c. avoid meaningful names
 - d. use nested IF only
6. Which assignment is more readable?
 - a. `v = d / t`
 - b. `x1 = a2 / b2`
 - c. `velocity = distance / time`
 - d. `a = b / c`
7. Accessing `arr[0]` is:
 - a. $O(n^2)$
 - b. $O(n \log n)$
 - c. $O(n)$
 - d. $O(1)$
8. Binary search on a sorted list is:
 - a. $O(\log n)$
 - b. $O(n^2)$
 - c. $O(1)$ always for any n
 - d. $O(n)$ because a loop exists
9. Bubble Sort and nested “compare every pair” loops are:
 - a. $O(1)$
 - b. $O(\log n)$
 - c. $O(n)$
 - d. $O(n^2)$
10. If n grows 10 times, a quadratic algorithm’s time grows about:
 - a. 2 times
 - b. 10 times
 - c. 100 times
 - d. log 10 times
11. LIFO is the rule of a:
 - a. queue
 - b. graph of cities
 - c. tree root only
 - d. stack
12. ENQUEUE adds an item at the:
 - a. Front of a queue
 - b. TOP of a stack only
 - c. Rear of a queue
 - d. root of a tree
13. A college organogram with a Principal at the top is modelled as a:
 - a. tree
 - b. queue
 - c. stack
 - d. contiguous array of roads
14. GPS maps (cities and roads, many possible routes) need a:
 - a. stack of undo actions
 - b. graph
 - c. strict one-parent tree only
 - d. single array index
15. To retrieve 87 from HEAD→12→98→23→87→56 you must:
 - a. jump with Array[87]
 - b. DEQUEUE the rear
 - c. follow four nodes from HEAD until 87 matches
 - d. POP from the bottom of a stack

Section B — Short questions ($10 \times 3 = 30$)

1. Define correctness of an algorithm. Name the two analysis methods used in this chapter.
2. List the components of a trace table. Why is a dry run useful for debugging?
3. Give four differences between a trace table and stepwise reasoning.
4. What is modularity? Explain SRP, coupling and abstraction in one sentence each.
5. State four ways to evaluate readability, with one example of a bad name and a good name.
6. Define Big O notation. Give one example each of $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$ and $O(n^2)$.
7. Write the three questions of the efficiency evaluation framework and answer them for searching 50 unsorted names.
8. List four refinements for clarity and four for efficiency.
9. Define data structure. Distinguish linear from non-linear with examples.
10. Define stack and queue. Include LIFO/FIFO, PUSH/POP, ENQUEUE/DEQUEUE, and one real-life example of each.

Section C — Long questions (attempt any $3 \times 10 = 30$)

1. The soldier-eligibility algorithm uses AllHeights = [1.91, 2.01, 1.87, 1.57, 1.48] and counts heights from 1.57 to 1.91 inclusive. (a) Draw a complete trace table. (b) State the printed Total. (c) Explain in stepwise-reasoning style why the boundaries 1.57 and 1.91 are counted. (d) Write equivalent Python and its output.
2. Copy the monoblock average algorithm for A = [1, 2, 3, 4, 5] and rewrite it with CalculateSum and FindAverage. (a) Show that SRP, loose coupling and abstraction are improved. (b) Dry-run both versions to the same average. (c) Give two readability improvements you would still make to the functions.
3. Using the growth table idea, (a) fill n = 8 and n = 32 for $O(1)$, $O(\log_2 n)$, $O(n)$, $O(n \log_2 n)$ and $O(n^2)$. (b) Explain what happens if n becomes 10 times larger for $O(n)$ and for $O(n^2)$. (c) A library has 1,000,000 sorted IDs. Choose linear or binary search using the Scale / Bottleneck / Worth-it framework. (d) Show a 4-by-4 nested loop counts 16 operations in Python.
4. Draw a labelled stack and a labelled queue. Then trace: (a) PUSH Data1, PUSH Data2, PUSH Data3, POP, POP. (b) ENQUEUE 55, 76, 90, D then DEQUEUE. Include Python that matches both traces. (c) Why is Undo a stack and a printer a queue?
5. A college needs structures for: (i) 200 integer marks, (ii) a playlist with frequent insertions, (iii) Undo in the fee-office editor, (iv) photocopier jobs, (v) the Principal–Dean–HOD chart, (vi) roads between Sindh cities. Identify each structure using the decision guide, draw the tree and the graph, and then trace retrieval of 20 from Array = [10, 50, 20, 70, 90, 30] and of 87 from the linked list 12→98→23→87→56→NULL.

Answer key. Finish the Chapter Exam first, then mark. Do not peek while the clock is running.

پہلے امتحان مکمل کریں، پھر اس کلید سے نمبر لگائیں۔

Check yourself — answers

1. **2.1 Analyze algorithms for correctness.** Answers: 1. When its output matches the expected results for a given input (and does so for all valid inputs). 2. Trace tables and stepwise reasoning. 3. Because a logical error can still produce output — the wrong output.
2. **2.1.1 Trace Tables.** Answers: 1. Line/step number, variables, input/output, condition check. 2. 1.57 is the lower boundary (inclusive, so True); 1.48 is below 1.57 (False). 3. 3 is printed; Step 6 runs three times (for 1.91, 1.87 and 1.57).
3. **2.1.2 Stepwise Reasoning.** Answers: 1. Logic verification. 2. Because no numbers have been added yet, so the running total should be zero. 3. The loop does not run and Sum remains 0, which is the correct sum of no numbers.
4. **2.1.3 Comparing Trace Tables and Stepwise Reasoning.** Answers: 1. Trace table. 2. It is very tedious and impractical (too many rows). 3. Stepwise reasoning (macro-level, conceptual / structural flaws).
5. **2.2 Evaluating the Clarity of an Algorithm.** Answers: 1. The algorithm is easy to read, understand and modify. 2. Modularity and readability. 3. Because a clear algorithm is easier to maintain, debug and extend over time.
6. **2.2.1 Modularity.** Answers: 1. One specific task. 2. CalculateSum (FindAverage only divides; the main program only prints). 3. All logic is in one block, so you cannot call “just the sum” from another algorithm.
7. **2.2.2 Readability.** Answers: 1. velocity = distance / time. 2. IF inside IF inside IF (or similar); it makes the flow hard to follow. 3. Any two of: consistent indentation, spacing, and casing (camelCase or snake_case).
8. **2.3 Assess Algorithm Efficiency.** Answers: 1. Time (speed) and space (memory). 2. Number of steps (Big O), number of conditions, number of repetitions.
9. **2.3.1 Number of Steps (Big O Notation).** Answers: 1. $O(1)$ arr[0]; $O(\log n)$ binary search; $O(n)$ linear search; $O(n^2)$ Bubble Sort / nested loops. 2. 1,024. 3. It makes about $\log n$ split-levels and does about n work on each level, so $n \times \log n$.
10. **2.3.2 Number of Conditions.** Answers: 1. Each condition must be evaluated True/False by the CPU. 2. A brute-force approach (and it reduces readability and speed). 3. AND short-circuits: the first part is False, so the division is never evaluated.
11. **2.3.3 Number of Repetitions.** Answers: 1. Poor — $O(n^2)$. 2. Each iteration halves the remaining problem, so the number of iterations is about $\log n$. 3. 25.
12. **2.3.4 The Efficiency Evaluation Framework.** Answers: 1. About 100× slower (10^2). 2. The most time-consuming section, usually the deepest loop. 3. Binary search is more complex to code and maintain, and for $n = 50$ the simplicity of $O(n)$ is practically better.
13. **2.4 Refinements to improve clarity and efficiency.** Answers: 1. average = totalSum / totalItems. 2. The already-sorted largest value at the end (shrink the inner loop). 3. Once, outside the loop, then reuse C inside.
14. **2.5 Concept of Data Structure.** Answers: 1. A systematic way of organising, storing and managing data for efficient access, modification and processing. 2. Linear: array, linked list, stack, queue. Non-linear: tree, graph. 3. It has exactly one predecessor and one successor.
15. **2.5.1 Linear Data Structures: Array.** Answers: 1. The items sit in one continuous block, next to each other. 2. 12. 3. Later elements must all be shifted right to make room.
16. **Linear Data Structures: Linked List.** Answers: 1. Data, and a link/pointer to the next node. 2. NULL (no next node). 3. Back follows the pointer to the previous page-node instead of jumping by index.
17. **Linear Data Structures: Stack.** Answers: 1. Last-In, First-Out. 2. Data3. 3. Each action is PUSHed; Undo POPs the most recent action first.
18. **Linear Data Structures: Queue.** Answers: 1. First-In, First-Out; added at Rear, removed from Front. 2. 55. 3. Documents must print in the order they arrived (first received, first printed), not last-received first.
19. **2.5.2 Non-Linear Data Structures: Tree.** Answers: 1. The root. 2. One-to-many parent-child. 3. A family tree, and folders inside folders (or a college organogram).
20. **Non-Linear Data Structures: Graph.** Answers: 1. Nodes/vertices and edges. 2. Social networks (Facebook friends) and GPS mapping. 3. A tree has a hierarchy and one root; a graph has no strict hierarchy and any node may connect to any other (cycles allowed).
21. **2.5.3 Operations on Data Structures.** Answers: 1. Insertion, deletion, traversal, searching, sorting. 2. Insertion (at the rear of a queue). 3. Traversal visits every item; searching stops when the target is found (or the structure ends).
22. **2.6 Identify data structures in problem scenarios.** Answers: 1. Array — known size, same type, fast index lookup. 2. Queue, because first-come, first-served. 3. Stack.
23. **2.7 Trace data retrieval in Array, List and Queue.** Answers: 1. 20. 2. Four (12, 98, 23, 87). 3. No — a queue only DEQUEUEs the Front, so A must leave first.
24. **Career Opportunities in Algorithms.** Answers: 1. Algorithm Engineer. 2. The choice of structure (array, queue) directly affects web performance.

Answer key

Multiple choice

1. b
2. a
3. d
4. b
5. a
6. c
7. d
8. a
9. d
10. c
11. d
12. c
13. a
14. b
15. c

Short questions — model answers

1. An algorithm is correct if its output matches the expected results for a given input (and does so for all valid inputs). The two methods are trace tables and stepwise reasoning (logic verification).
2. Components: line/step number, a column per variable, input/output, and a condition-check column. A dry run acts as the computer, so you see logical errors where the program runs but the values go wrong, and you can validate that the intention is met.
3. Trace table: small blocks/complex loops/beginners; high effort (bad for 1,000 iterations); catches arithmetic/line mistakes; micro-level exact values. Stepwise reasoning: large systems and proofs; moderate effort; catches conceptual/structural flaws; macro-level overall state.
4. Modularity is breaking an algorithm into smaller independent parts (functions, classes, modules). SRP: each module does one task. Coupling: modules should be loosely coupled so a change in one does not break others. Abstraction: hide inner details (call Sort without knowing Bubble vs Selection).
5. Naming conventions, self-documentation, simple control flow, consistency of indentation/spacing/casing. Bad: $v = d / t$. Good: `velocity = distance / time`.
6. Big O describes how time or memory grows as n grows. $O(1)$: `arr[0]`; $O(\log n)$: binary search; $O(n)$: linear search; $O(n \log n)$: Merge Sort; $O(n^2)$: Bubble Sort / nested loops.
7. Does it scale? $n = 50$ is tiny, even $O(n)$ is fine. Bottleneck? A single scan of 50 names. Worth it? Binary search needs sorted data and more complex code, so linear search is practically better.
8. Clarity: meaningful names; modules; structure/indentation; short why-comments. Efficiency: reduce extra steps (shrink Bubble Sort's inner loop); better algorithms (binary vs linear on sorted millions); avoid re-computing (cache $C = A+B$ outside the loop); appropriate data structures (array for direct access, tree for hierarchy).
9. A data structure is a systematic way of organising, storing and managing data for efficient access, modification and processing. Linear: sequence with one predecessor and one successor (array, linked list, stack, queue). Non-linear: hierarchical or network links (tree, graph).
10. Stack: LIFO; PUSH adds at TOP; POP removes from TOP; example Undo. Queue: FIFO; ENQUEUE adds at Rear; DEQUEUE removes from Front; example printer jobs.

Long questions — model answers

1. (a) Columns: Step, Total, Height, AllHeights, condition, output. Step 2 sets Total = 0. Step 3 stores the five heights. Then five loop cycles:

- 1.91 → True → Total 1
- 2.01 → False → Total stays 1
- 1.87 → True → Total 2
- 1.57 → True (boundary) → Total 3
- 1.48 → False → Total stays 3

(b) Printed Total = **3**.

(c) The condition is height ≥ 1.57 AND height ≤ 1.91 , so the closed interval includes both ends. 1.57 and 1.91 satisfy the inequalities; 2.01 and 1.48 do not. Initial Total = 0 is valid; each True branch adds one; False skips the add; after five items Total is the count of eligible soldiers.

(d) Python:

```
AllHeights = [1.91, 2.01, 1.87, 1.57, 1.48]
Total = 0
for height in AllHeights:
    if height >= 1.57 and height <= 1.91:
        Total = Total + 1
print(Total)
```

OUTPUT
3

2. (a) Monoblock: one sequence does the loop, the division and the print, so you cannot reuse the sum. Modular version: CalculateSum only adds; FindAverage only divides; main only prints. SRP holds. CalculateSum can be called from another program (loose coupling). Main does not see the loop (abstraction).

(b) Both compute total = 15 and avg = $15/5 = 3.0$.

```
def calculate_sum(A, n):
    total = 0
    for i in range(n):
        total = total + A[i]
    return total

def find_average(A, n):
    return calculate_sum(A, n) / n

print(find_average([1, 2, 3, 4, 5], 5))
```

OUTPUT
3.0

(c) Keep snake_case names; avoid 1-based I in Python; a one-line comment on why you divide by n (empty-list guard could be added later) is enough — do not comment every addition.

3. (a) Growth ($\log_2$):

n	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$
8	1	3	8	24	64
32	1	5	32	160	1,024

(b) $O(n)$: 10× input → about 10× time. $O(n^2)$: 10× input → about 100× time, which fails for large sets.

(c) Scale: a million IDs; linear worst case is a million checks, binary about 20. Bottleneck: the search. Worth it: the IDs are already sorted and binary search is standard, so choose **binary search** $O(\log n)$.

(d)

```
n = 4
count = 0
for i in range(n):
    for j in range(n):
        count = count + 1
print(count)
```

OUTPUT
16

4. Draw the vertical stack with TOP on Data3 after three PUSHes, and the horizontal queue with FRONT on 55 and REAR on 90 (then D).

(a) Stack: [] → [Data1] → [Data1, Data2] → [Data1, Data2, Data3] → POP Data3 → POP Data2 → [Data1].

```
stack = []
stack.append("Data1")
stack.append("Data2")
stack.append("Data3")
print(stack.pop())
print(stack.pop())
print(stack)
```

OUTPUT
Data3
Data2
['Data1']

(b) Queue: [55] → [55, 76] → [55, 76, 90] → [55, 76, 90, D] → DEQUEUE 55 → [76, 90, D].

```
from collections import deque
q = deque()
q.append(55)
q.append(76)
q.append(90)
q.append("D")
print(q.popleft())
print(list(q))
```

OUTPUT
55
[76, 90, 'D']

(c) Undo must cancel the most recent action (LIFO → stack). A printer must respect arrival order (FIFO → queue).

5. (i) Array — known count, same type, index lookup. (ii) Linked list — next-pointer, cheap insert. (iii) Stack — reverse order of entry. (iv) Queue — first-come, first-served. (v) Tree — parent-child from the Principal root (Administrative Head / Academic Dean, then Accounts, HR, HOD Math, HOD CS). (vi) Graph — cities as vertices, roads as edges, alternative routes.

Array retrieval: Array[2] = 20 in one direct access (or after visiting 10 then 50 then 20 if you scan).

List retrieval: Step1 12 ≠ 87, Step2 98 ≠ 87, Step3 23 ≠ 87, Step4 87 = 87 FOUND True.

Self-assessment: I can...

- ☐ prove correctness with a full trace table, including boundary values.
- ☐ judge modularity (SRP, coupling, abstraction) and readability (N-S-S-C).
- ☐ apply the Scale / Bottleneck / Worth-it framework, including the $n = 50$ warning.
- ☐ define a data structure and classify linear vs non-linear.
- ☐ draw a tree and a graph and pick a structure from a scenario using the decision guide.
- ☐ write stepwise reasoning (start, loop, finish, edges) and compare it with a trace table.
- ☐ state $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$ and $O(n^2)$ with examples and a growth table.
- ☐ refine an algorithm for names, modules, fewer loop steps and cached values.
- ☐ draw and trace an array, linked list, stack (PUSH/POP) and queue (ENQUEUE/DEQUEUE).
- ☐ trace retrieval: array jump, list walk from HEAD, queue Front only.